Client Requirements Document

Client: Promtior

Initial Date: 20/04/2026

Objective: Develop and deploy a chatbot assistant that uses the RAG architecture to answer questions about the content of the Promtior website and other documents specified by the client.

System Specification

Functionalities

The Chatbot assistant should be able to answer the following questions as a minimum:

1. What services does Promtior offer?
2. When was the company founded?

Technologies to be Used

1. Implement the solution using LangChain with Langserve
2. Use the OpenAI API
3. Deploy on a cloud (AWS, Azure, other)

Deliverables

1. Documentation:
    a. Project Overview: A brief summary in your own words about how you approached and solved the challenge. This includes the implementation logic, the main challenges encountered, and how you overcame them.
    b. Component Diagram: A diagram showing the components involved in the solution and their interactions from the time the question is received by the chatbot until the response is given.
2. Source Code

Documentation

Proposed Solution

The chatbot assistant will be designed to answer questions related exclusively to the content of the Promtior website and any uploaded documentation.

Functional Scope

- The chatbot will only provide answers based on the available documentation and website content.
- It will not generate or infer information that is not explicitly present in these sources.
- Responses will be strictly limited to the scope of the provided materials.
- If the requested information is not found, the chatbot will clearly indicate that the information is unavailable or not applicable.
- If a user query cannot be understood, the chatbot will notify the user and provide suggestions for clearer or alternative questions.

Document Processing

- The system will process and transform the website content and documents into a format suitable for retrieval and use by the chatbot.
- A Document processing pipeline will be implemented.

Conversation

- The chatbot will operate in a stateful manner, storing user conversations.
- If a conversation exceeds a certain length, it will be summarized and the summary stored to maintain context efficiently.
- Each user will be identified via a token, allowing their session and conversation history to persist even after refreshing the page.

Restrictions

- Minimize Tokens Without Hurting Performance

Project Overview

A chatbot for public users was designed to answer questions about Promtior based on website content and provided documents. In addition, a pipeline was developed to automatically process new documents and integrate them into the system.

The prompt sent to the OpenAI API consisted of four parts:

- System instructions: How the assistant should behave and constraining responses to the provided source
- User question: Current query
- Document chunks: information selected through vector search to support the answer
- Message history to improve user experience.

These prompt components were iterated and changed based on observed results. Vectorization improved the chatbot's performance, by choosing relevant information more efficiently, reducing unnecessary context, and reducing token usage**.**

Implementation of Solution

Document Processing

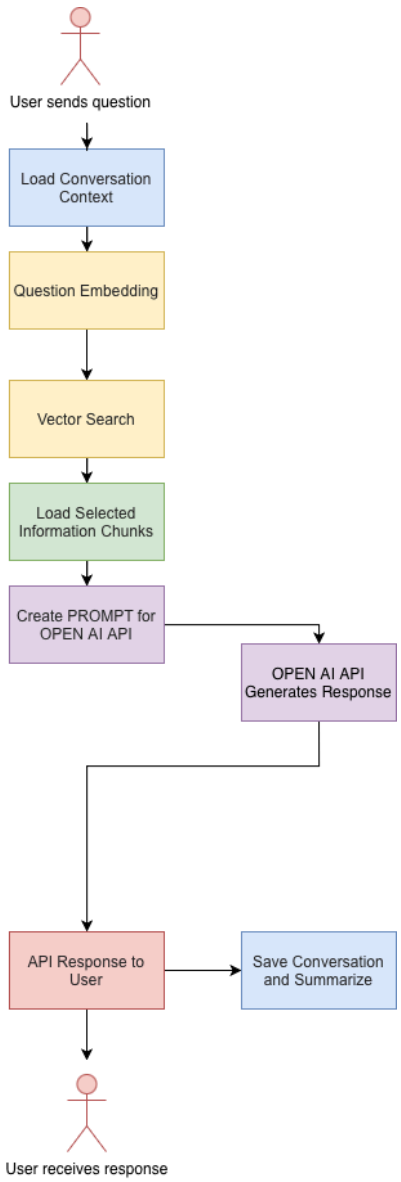
RAG requires preprocessing of the information that the chatbot will use. The pipeline flow is shown in the diagram.

1. Raw Document: Web pages (HTML) and PDF documents.
2. Text Extraction & Cleaning: Content is extracted from HTML and PDFs, noise is removed, and metadata is identified.
3. Chunking: The cleaned text is split into overlapping chunks, trying to preserve paragraphs and phrases together.
4. Embedding Generation: For each chunk, a vector embedding is generated by calling an OpenAI embedding model.
5. Store: Each embedding vector is stored in RDS together with its corresponding text chunk and associated metadata

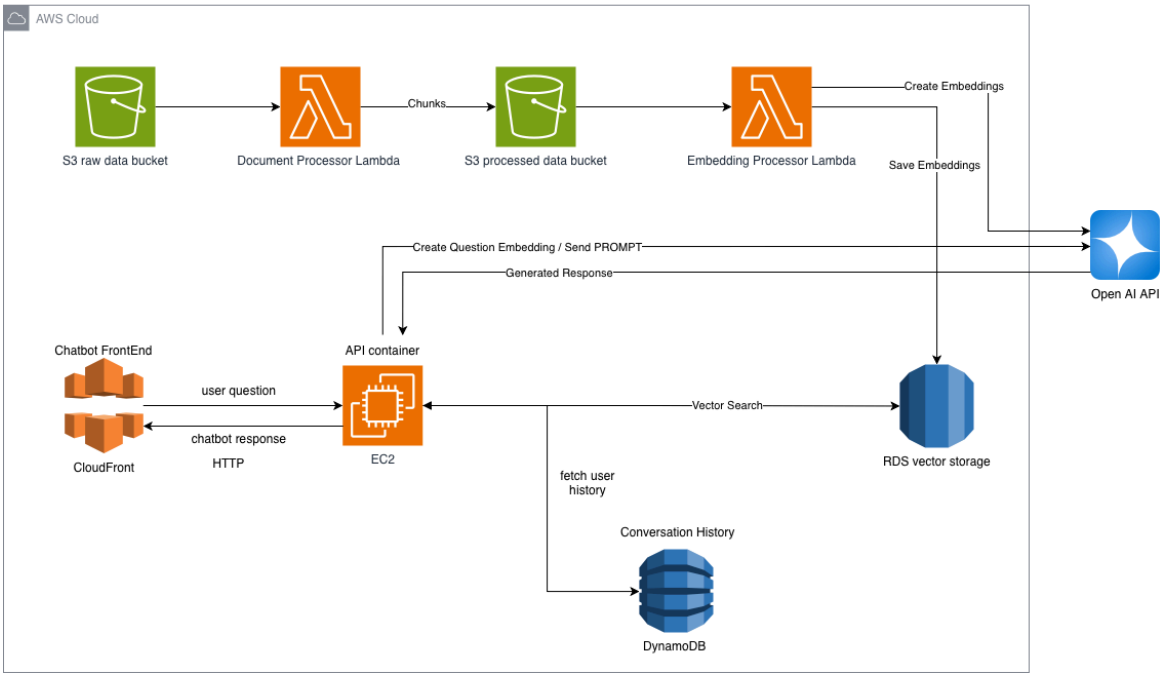
RAG

1. User Sends Question through the chatbot interface
2. Load Conversation Context: If previous interaction exist, they are loaded from DB
3. Question Encoding: The query is converted into an embedding vector using OPEN AI Embedding Model.
4. Vector Search: The query vector is compared against the stored document embeddings to identify similar content.

5. Load Selected Chunk Selection: The most relevant information chunks are retrieved from the vector database to support the answer.
6. Create Prompt for OPEN AI API: A prompt is assembled containing system instructions, the user question, retrieved chunks, and recent message history.
7. OpenAI API Generates Response: The LLM uses the prompt context to generate an answer grounded in the information available and according to the instructions given.
8. Response Returned to User: The generated response is sent back through the chatbot interface.
9. Save Conversation and Summarize: Both question and reply are stored to preserve conversational context for future questions. If history becomes too long, it is summarized.



Deployment



Components

- **EC2**: Deployed within a VPC, hosts the backend application where the LangServe API runs and handles chatbot requests.
- **DynamoDB**: Stores the conversation history and related session data for the chatbot.
- **CloudFront**: Serves the chat frontend and delivers it to users.
- **API Communication:** The frontend communicates with the backend through HTTP REST requests sent from the CloudFront-served chat application to the LangServe API running on EC2.
- **OpenAI API**: Called to generate embeddings, final chatbot answers and, when needed, to summarize long conversation history.
- **RDS:** Stores the RAG knowledge base. For each document chunk, it stores the embedding vector, the chunk text, and its metadata
- **S3:** Two buckets are used one for storing raw data and another for storing processed data after transformation in chunks.
- **Lambda**:
    - Document Processing: Triggered when a new document is uploaded to the raw S3 bucket.
    - Embedding: Triggered when new processed chunks are created.